Linear Data Structures

沈威宇

November 8, 2026

1 Linear Data Structures

I Unordered list

An unordered list is a list where each element holds a relative position with respect to the others. Each element is called a node. If applicable, the first node is called head and the last node is called tail.

II Ordered list

An ordered list is a list which are ordered based upon some underlying characteristic of the elements.

III Stack

A stack is a last-in-first-out (LIFO) list. Adding an element is called push and is of time complexity O(1), the last pushed element is called top, and the first pushed element is called bottom. Removing and returning top is called pop and is of time complexity O(1). Peeking top is called top and is of time complexity O(1).

To keep track of the minimum element of a stack in O(1) time with O(n) extra space. One can maintain the original stack of elements and an extra stack of extrema. When an element is pushed to the original stack, it is compared with the top of the extrema stack and the minimum one is pushed to the extrema stack (for the first element, it is pushed to the extrema stack). When an element is popped from the original stack, an element is popped from the extrema stack. This way, the top of the extrema stack is always the minimum of the elements in the original stack.

IV Queue

A queue is first-in-first-out (FIFO) list. Adding an element is called enqueue and is of time complexity O(1), the first enqueued element is called front, and the last enqueued element is called back. Removing and returning front is called dequeue and is of time complexity O(1). Peeking front is called peek and is of time complexity O(1).

V Array

An array is a fixed-size list of elements of the same memory size, each identified by at least one index, stored contiguously such that the position (memory address) of each element can be computed from its index in O(1) time, i.e., random access in O(1) time.

VI Dynamic array, growable array, resizable array, dynamic table, mutable array, or array list

A dynamic array is a variable-size list that stores elements in an array of a capacity and reserves an array of new capacity elsewhere and moves to it whenever its size reaches its capacity. It has random access and popping back in O(1) time, pushing back in amortized O(1) time, but removal and insertion in O(n) time. It is a stack with front as bottom and back as top.

An implementation in C++:

```
#pragma once // vector.hpp

#ifndef _VECTOR_GROW
#define _VECTOR_GROW 2
#endif

#include <algorithm>
#include <compare>
#include <cstddef>
#include <cstring>
#include <functional>
#include <initializer_list>
#include <iterator>
#include <limits>
#include <memory>
#include <stdexcept>
#include <utility>

template <class T, class Allocator = std::allocator<T>>
  requires std::is_same_v<T, typename Allocator::value_type>
class vector {
public:
  using value_type = T;
  using allocator_type = Allocator;
  using size_type = std::size_t;
  using difference_type = std::ptrdiff_t;
  using reference = T &;
  using const_reference = const T &;
  using pointer = typename std::allocator_traits<Allocator>::pointer;
  using const_pointer =
      typename std::allocator_traits<Allocator>::const_pointer;
  using iterator = T *;
  using const_iterator = const T *;
  using reverse_iterator = std::reverse_iterator<iterator>;
  using const_reverse_iterator = std::reverse_iterator<const_iterator>;

private:
  [[no_unique_address]] Allocator alloc;
  T *elems;
  std::size_t sz;
  std::size_t cap;

  constexpr void deallocate() {
    if (elems) {
      std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
      elems = nullptr;
    }
```

```
    sz = cap = 0;
  }

  constexpr void destroy_deallocate() {
    if (elems) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = 0; i < sz; ++i)
          std::allocator_traits<Allocator>::destroy(alloc, elems + i);
      std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
      elems = nullptr;
    }
    cap = 0;
  }

  template <bool UseMove>
  constexpr void transfer_elements(T *new_elems, std::size_t count) {
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::memcpy(new_elems, elems, count * sizeof(T));
    } else {
      std::size_t i = 0;
      try {
        for (; i < count; ++i) {
          if constexpr (UseMove) {
            std::allocator_traits<Allocator>::construct(alloc, new_elems + i,
                                                        std::move(elems[i]));
          } else {
            std::allocator_traits<Allocator>::construct(alloc, new_elems + i,
                                                        elems[i]);
          }
        }
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          std::allocator_traits<Allocator>::destroy(alloc, new_elems + j);
        throw;
      }
    }
  }

public:
  constexpr vector() noexcept(noexcept(Allocator()))
      : alloc(Allocator()), elems(nullptr), sz(0), cap(0) {}
  explicit constexpr vector(const Allocator &alloc_) noexcept
      : alloc(alloc_), elems(nullptr), sz(0), cap(0) {}

  explicit vector(std::size_t count, const Allocator &alloc_ = Allocator())
      : alloc(alloc_), sz(count), cap(count) {
    if (count == 0)
      elems = nullptr;
    else {
      elems = std::allocator_traits<Allocator>::allocate(alloc, count);
      if constexpr (!std::is_trivially_copyable_v<T>) {
        std::size_t i = 0;
        try {
          for (; i < count; ++i)
            std::allocator_traits<Allocator>::construct(alloc, elems + i);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, elems + j);
          deallocate();
          throw;
        }
      }
```

```
110     }
111   }
112
113   constexpr vector(std::size_t count, const T &value,
114                    const Allocator &alloc_ = Allocator())
115       : alloc(alloc_), sz(count), cap(count) {
116     if (count == 0)
117       elems = nullptr;
118     else {
119       elems = std::allocator_traits<Allocator>::allocate(alloc, count);
120       if constexpr (std::is_trivially_copyable_v<T>)
121         std::fill_n(elems, count, value);
122       else {
123         std::size_t i = 0;
124         try {
125           for (; i < count; ++i)
126             std::allocator_traits<Allocator>::construct(alloc, elems + i,
127                                                         value);
128         } catch (...) {
129           for (std::size_t j = 0; j < i; ++j)
130             std::allocator_traits<Allocator>::destroy(alloc, elems + j);
131           deallocate();
132           throw;
133         }
134       }
135     }
136   }
137
138   template <std::input_iterator InputIt>
139   constexpr vector(InputIt first, InputIt last,
140                    const Allocator &alloc_ = Allocator())
141       : alloc(alloc_), elems(nullptr), sz(0), cap(0) {
142     if constexpr (std::forward_iterator<InputIt>) {
143       if (first == last)
144         return;
145       std::size_t count = static_cast<std::size_t>(std::distance(first, last));
146       if (count > 0) {
147         elems = std::allocator_traits<Allocator>::allocate(alloc, count);
148         sz = cap = count;
149         if constexpr (std::is_trivially_copyable_v<T>) {
150           if (std::contiguous_iterator<InputIt>)
151             std::memmove(elems, &*first, count * sizeof(T));
152           else
153             std::copy(first, last, elems);
154         } else {
155           std::size_t i = 0;
156           try {
157             for (; first != last; ++first, ++i)
158               std::allocator_traits<Allocator>::construct(alloc, elems + i,
159                                                           *first);
160           } catch (...) {
161             for (std::size_t j = 0; j < i; ++j)
162               std::allocator_traits<Allocator>::destroy(alloc, elems + j);
163             deallocate();
164             throw;
165           }
166         }
167       }
168     } else
169       for (; first != last; ++first)
170         push_back(*first);
171   }
```

```
172
173   constexpr vector(const vector &other)
174       : alloc(std::allocator_traits<
175               Allocator>::select_on_container_copy_construction(other.alloc)),
176         elems(nullptr), sz(other.sz), cap(other.cap) {
177     if (cap > 0) {
178       elems = std::allocator_traits<Allocator>::allocate(alloc, cap);
179       if constexpr (std::is_trivially_copyable_v<T>)
180         std::memmove(elems, other.elems, sz * sizeof(T));
181       else {
182         std::size_t i = 0;
183         try {
184           for (; i < sz; ++i)
185             std::allocator_traits<Allocator>::construct(alloc, elems + i,
186                                                         other.elems[i]);
187         } catch (...) {
188           for (std::size_t j = 0; j < i; ++j)
189             std::allocator_traits<Allocator>::destroy(alloc, elems + j);
190           std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
191           throw;
192         }
193       }
194     }
195   }
196
197   constexpr vector(vector &&other) noexcept(
198       std::allocator_traits<
199           Allocator>::propagate_on_container_move_assignment::value ||
200       std::allocator_traits<Allocator>::is_always_equal::value)
201       : alloc(std::move(other.alloc)), elems(nullptr), sz(0), cap(0) {
202     if constexpr (std::allocator_traits<Allocator>::
203                       propagate_on_container_move_assignment::value ||
204                   std::allocator_traits<Allocator>::is_always_equal::value ||
205                   (requires {
206                     { alloc == other.alloc } -> std::convertible_to<bool>;
207                   } && (alloc == other.alloc))) {
208       elems = std::exchange(other.elems, nullptr);
209       sz = std::exchange(other.sz, 0);
210       cap = std::exchange(other.cap, 0);
211     } else {
212       if (other.sz > 0) {
213         elems = std::allocator_traits<Allocator>::allocate(alloc, other.sz);
214         sz = cap = other.sz;
215         if constexpr (std::is_trivially_move_constructible_v<T>)
216           std::memmove(elems, other.elems, sz * sizeof(T));
217         else if (std::is_nothrow_move_constructible_v<T> ||
218                  !std::is_copy_constructible_v<T>) {
219           std::size_t i = 0;
220           try {
221             for (; i < sz; ++i)
222               std::allocator_traits<Allocator>::construct(
223                   alloc, elems + i, std::move(other.elems[i]));
224           } catch (...) {
225             for (std::size_t j = 0; j < i; ++j)
226               std::allocator_traits<Allocator>::destroy(alloc, elems + j);
227             std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
228             throw;
229           }
230         } else {
231           std::size_t i = 0;
232           try {
233             for (; i < sz; ++i)
```

```
            std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        other.elems[i]);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, elems + j);
          std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
          throw;
        }
      }
      if constexpr (!std::is_trivially_destructible_v<T> &&
                    !std::is_trivially_move_constructible_v<T>) {
        if (std::is_nothrow_move_constructible_v<T> ||
            !std::is_copy_constructible_v<T>) {
          for (std::size_t j = 0; j < sz; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, other.elems + j);
        }
      }
      other.elems = nullptr;
      other.sz = other.cap = 0;
    }
  }
}

constexpr vector(const vector &other, const Allocator &alloc_)
    : alloc(alloc_), elems(nullptr), sz(other.sz), cap(other.cap) {
  if (cap > 0) {
    elems = std::allocator_traits<Allocator>::allocate(alloc, cap);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::memmove(elems, other.elems, sz * sizeof(T));
    else {
      std::size_t i = 0;
      try {
        for (; i < sz; ++i)
          std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                      other.elems[i]);
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          std::allocator_traits<Allocator>::destroy(alloc, elems + j);
        std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
        throw;
      }
    }
  }
}

vector(std::initializer_list<T> ilist, const Allocator &alloc_ = Allocator())
    : alloc(alloc_), elems(nullptr), sz(ilist.size()), cap(ilist.size()) {
  if (cap > 0) {
    elems = std::allocator_traits<Allocator>::allocate(alloc, cap);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::memmove(elems, ilist.begin(), sz * sizeof(T));
    else {
      std::size_t i = 0;
      try {
        for (const T *j = ilist.begin(); j != ilist.end(); ++j, ++i)
          std::allocator_traits<Allocator>::construct(alloc, elems + i, *j);
      } catch (...) {
        for (std::size_t k = 0; k < i; ++k)
          std::allocator_traits<Allocator>::destroy(alloc, elems + k);
        std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
        throw;
      }
```

```
    }
  }
}

constexpr ~vector() {
  if (elems) {
    if constexpr (!std::is_trivially_destructible_v<T>)
      for (std::size_t i = 0; i < sz; ++i)
        std::allocator_traits<Allocator>::destroy(alloc, elems + i);
    std::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
  }
}

constexpr vector &operator=(const vector &other) {
  if (this != &other) {
    if constexpr (std::allocator_traits<Allocator>::
                      propagate_on_container_copy_assignment::value &&
                  (requires {
                    { alloc != other.alloc } -> std::convertible_to<bool>;
                  } && (alloc != other.alloc))) {
      destroy_deallocate();
      alloc = other.alloc;
      if (cap < other.sz) {
        elems = std::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      }
    } else if (other.sz > cap) {
      if (elems)
        destroy_deallocate();
      elems = std::allocator_traits<Allocator>::allocate(alloc, other.sz);
      cap = other.sz;
    } else if constexpr (!std::is_trivially_destructible_v<T>)
      for (std::size_t i = other.sz; i < sz; ++i)
        std::allocator_traits<Allocator>::destroy(alloc, elems + i);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::memcpy(elems, other.elems, other.sz * sizeof(T));
    else {
      std::size_t i = 0;
      try {
        for (; i < other.sz; ++i) {
          if (i < sz)
            elems[i] = other.elems[i];
          else
            std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        other.elems[i]);
        }
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          std::allocator_traits<Allocator>::destroy(alloc, elems + j);
        throw;
      }
    }
    sz = other.sz;
  }
  return *this;
}

constexpr vector &operator=(vector &&other) noexcept(
    std::allocator_traits<
        Allocator>::propagate_on_container_move_assignment::value ||
    std::allocator_traits<Allocator>::is_always_equal::value) {
  if (this != &other) {
```

```
    if constexpr (std::allocator_traits<Allocator>::
                      propagate_on_container_move_assignment::value ||
                  std::allocator_traits<Allocator>::is_always_equal::value ||
                  (requires {
                    { alloc == other.alloc } -> std::convertible_to<bool>;
                  } && (alloc == other.alloc))) {
      destroy_deallocate();
      alloc = std::move(other.alloc);
      elems = std::exchange(other.elems, nullptr);
      sz = std::exchange(other.sz, 0);
      cap = std::exchange(other.cap, 0);
    } else {
      if (other.sz > cap) {
        if (elems)
          destroy_deallocate();
        elems = std::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      } else if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = other.sz; i < sz; ++i)
          std::allocator_traits<Allocator>::destroy(alloc, elems + i);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memcpy(elems, other.elems, other.sz * sizeof(T));
      else if (std::is_nothrow_move_constructible_v<T> ||
               !std::is_copy_constructible_v<T>) {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            std::allocator_traits<Allocator>::construct(
                alloc, elems + i, std::move(other.elems[i]));
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      } else {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        other.elems[i]);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      }
      sz = other.sz;
    }
  }
  return *this;
}

constexpr vector &operator=(std::initializer_list<T> ilist) {
  assign(ilist.begin(), ilist.end());
  return *this;
}

constexpr void assign(std::size_t count, const T &value) {
  if (count > cap) {
    destroy_deallocate();
    sz = 0;
    elems = std::allocator_traits<Allocator>::allocate(alloc, count);
```

```
      cap = count;
    }
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::fill_n(elems, count, value);
    } else {
      std::size_t min_size = std::min(sz, count);
      for (std::size_t i = 0; i < min_size; ++i)
        elems[i] = value;
      for (std::size_t i = min_size; i < count; ++i)
        std::allocator_traits<Allocator>::construct(alloc, elems + i, value);
      if (sz > count)
        for (std::size_t i = count; i < sz; ++i)
          std::allocator_traits<Allocator>::destroy(alloc, elems + i);
    }
    sz = count;
  }

  template <std::input_iterator InputIt>
  constexpr void assign(InputIt first, InputIt last) {
    if constexpr (std::forward_iterator<InputIt>) {
      std::size_t count = static_cast<std::size_t>(std::distance(first, last));
      if (count > cap) {
        destroy_deallocate();
        sz = 0;
        elems = std::allocator_traits<Allocator>::allocate(alloc, count);
        cap = count;
      }
      if constexpr (std::is_trivially_copyable_v<T>) {
        if constexpr (std::contiguous_iterator<InputIt>) {
          std::memmove(elems, &*first, count * sizeof(T));
        } else {
          std::copy(first, last, elems);
        }
      } else {
        std::size_t i = 0;
        for (; i < count && first != last; ++i, ++first) {
          if (i < sz)
            elems[i] = *first;
          else
            std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        *first);
        }
        if (sz > count)
          for (std::size_t j = count; j < sz; ++j)
            std::allocator_traits<Allocator>::destroy(alloc, elems + j);
      }
      sz = count;
    } else {
      clear();
      for (; first != last; ++first)
        push_back(*first);
    }
  }

  constexpr void assign(std::initializer_list<T> ilist) {
    assign(ilist.begin(), ilist.end());
  }

  constexpr allocator_type get_allocator() const noexcept { return alloc; }

  constexpr T &at(std::size_t index) {
    if (index >= sz)
```

```
482       throw std::out_of_range("vector");
483     return elems[index];
484   }
485
486   constexpr const T &at(std::size_t index) const {
487     if (index >= sz)
488       throw std::out_of_range("vector");
489     return elems[index];
490   }
491
492   constexpr T &operator[](std::size_t index) { return elems[index]; }
493   constexpr const T &operator[](std::size_t index) const {
494     return elems[index];
495   }
496   constexpr T &front() { return *elems; }
497   constexpr const T &front() const { return *elems; }
498   constexpr T &back() { return elems[sz - 1]; }
499   constexpr const T &back() const { return elems[sz - 1]; }
500
501   constexpr T *data() noexcept { return elems; }
502   constexpr const T *data() const noexcept { return elems; }
503
504   constexpr iterator begin() noexcept { return elems; }
505   constexpr const_iterator begin() const noexcept { return elems; }
506   constexpr const_iterator cbegin() const noexcept { return elems; }
507
508   constexpr iterator end() noexcept { return elems + sz; }
509   constexpr const_iterator end() const noexcept { return elems + sz; }
510   constexpr const_iterator cend() const noexcept { return elems + sz; }
511
512   constexpr reverse_iterator rbegin() noexcept {
513     return reverse_iterator(end());
514   }
515   constexpr const_reverse_iterator rbegin() const noexcept {
516     return const_reverse_iterator(cend());
517   }
518   constexpr const_reverse_iterator crbegin() const noexcept {
519     return const_reverse_iterator(cend());
520   }
521
522   constexpr reverse_iterator rend() noexcept {
523     return reverse_iterator(begin());
524   }
525   constexpr const_reverse_iterator rend() const noexcept {
526     return const_reverse_iterator(cbegin());
527   }
528   constexpr const_reverse_iterator crend() const noexcept {
529     return const_reverse_iterator(cbegin());
530   }
531
532   constexpr bool empty() const noexcept { return sz == 0; }
533   constexpr std::size_t size() const noexcept { return sz; }
534   constexpr std::size_t max_size() const noexcept {
535     return std::min(
536         std::allocator_traits<Allocator>::max_size(alloc),
537         static_cast<std::size_t>(std::numeric_limits<std::ptrdiff_t>::max()));
538   }
539
540   constexpr void reserve(std::size_t new_cap) {
541     if (new_cap <= cap)
542       return;
543     if (new_cap > max_size())
```

```
544       throw std::length_error("vector");
545     T *new_elems = std::allocator_traits<Allocator>::allocate(alloc, new_cap);
546     if (elems) {
547       if constexpr (std::is_trivially_copyable_v<T>) {
548         std::memcpy(new_elems, elems, sz * sizeof(T));
549       } else if (std::is_nothrow_move_constructible_v<T> ||
550                  !std::is_copy_constructible_v<T>) {
551         transfer_elements<true>(new_elems, sz);
552       } else {
553         transfer_elements<false>(new_elems, sz);
554       }
555       destroy_deallocate();
556     }
557     elems = new_elems;
558     cap = new_cap;
559   }
560
561   constexpr std::size_t capacity() const noexcept { return cap; }
562
563   constexpr void shrink_to_fit() {
564     if (cap != sz) {
565       if (sz == 0)
566         destroy_deallocate();
567       else {
568         T *new_elems = std::allocator_traits<Allocator>::allocate(alloc, sz);
569         if constexpr (std::is_trivially_copyable_v<T>) {
570           std::memcpy(new_elems, elems, sz * sizeof(T));
571         } else if (std::is_nothrow_move_constructible_v<T> ||
572                    !std::is_copy_constructible_v<T>) {
573           transfer_elements<true>(new_elems, sz);
574         } else {
575           transfer_elements<false>(new_elems, sz);
576         }
577         destroy_deallocate();
578         elems = new_elems;
579         cap = sz;
580       }
581     }
582   }
583
584   constexpr void clear() noexcept {
585     if constexpr (!std::is_trivially_destructible_v<T>)
586       for (std::size_t i = 0; i < sz; ++i)
587         std::allocator_traits<Allocator>::destroy(alloc, elems + i);
588     sz = 0;
589   }
590
591   constexpr iterator insert(const_iterator pos, const T &value) {
592     std::size_t index = pos - elems;
593     if (index > sz) {
594       if (index >= cap)
595         reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + 1));
596       for (std::size_t i = sz; i < index; ++i) {
597         std::allocator_traits<Allocator>::construct(alloc, elems + i);
598       }
599       sz = index;
600     } else if (sz >= cap)
601       reserve(sz ? sz * _VECTOR_GROW : 1);
602     if (index < sz) {
603       if constexpr (std::is_trivially_copyable_v<T>) {
604         std::memmove(elems + index + 1, elems + index,
605                      (sz - index) * sizeof(T));
```

```
606         elems[index] = value;
607       } else {
608         std::allocator_traits<Allocator>::construct(alloc, elems + sz,
609                                                     std::move(elems[sz - 1]));
610         for (std::size_t i = sz - 1; i > index; --i)
611           elems[i] = std::move(elems[i - 1]);
612         elems[index] = value;
613       }
614     } else {
615       std::allocator_traits<Allocator>::construct(alloc, elems + index, value);
616     }
617     ++sz;
618     return elems + index;
619   }
620
621   constexpr iterator insert(const_iterator pos, T &&value) {
622     std::size_t index = pos - elems;
623     if (index > sz) {
624       if (index >= cap)
625         reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + 1));
626       for (std::size_t i = sz; i < index; ++i) {
627         std::allocator_traits<Allocator>::construct(alloc, elems + i);
628       }
629       sz = index;
630     }
631     if (sz >= cap)
632       reserve(sz ? sz * _VECTOR_GROW : 1);
633     if (index < sz) {
634       if constexpr (std::is_trivially_copyable_v<T>) {
635         std::memmove(elems + index + 1, elems + index,
636                      (sz - index) * sizeof(T));
637         elems[index] = std::move(value);
638       } else {
639         std::allocator_traits<Allocator>::construct(alloc, elems + sz,
640                                                     std::move(elems[sz - 1]));
641         for (std::size_t i = sz - 1; i > index; --i)
642           elems[i] = std::move(elems[i - 1]);
643         elems[index] = std::move(value);
644       }
645     } else {
646       std::allocator_traits<Allocator>::construct(alloc, elems + index,
647                                                   std::move(value));
648     }
649     ++sz;
650     return elems + index;
651   }
652
653   constexpr iterator insert(const_iterator pos, std::size_t count,
654                             const T &value) {
655     if (count == 0)
656       return const_cast<iterator>(pos);
657     std::size_t index = pos - elems;
658     if (index > sz) {
659       if (index >= cap)
660         reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + count));
661       for (std::size_t i = sz; i < index; ++i) {
662         std::allocator_traits<Allocator>::construct(alloc, elems + i);
663       }
664       sz = index;
665     }
666     if (sz + count > cap)
667       reserve(std::max(sz ? sz * _VECTOR_GROW : 1, sz + count));
```

```
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::memmove(elems + index + count, elems + index,
                   (sz - index) * sizeof(T));
      std::fill_n(elems + index, count, value);
    } else {
      for (std::size_t i = sz + count - 1; i >= index + count; --i) {
        if (i < sz)
          elems[i] = std::move(elems[i - count]);
        else
          std::allocator_traits<Allocator>::construct(
              alloc, elems + i, std::move(elems[i - count]));
      }
      for (std::size_t i = index; i < index + count; ++i) {
        if (i < sz)
          elems[i] = value;
        else
          std::allocator_traits<Allocator>::construct(alloc, elems + i, value);
      }
    }
    sz += count;
    return elems + index;
  }

  template <std::input_iterator InputIt>
  constexpr iterator insert(const_iterator pos, InputIt first, InputIt last) {
    std::size_t index = pos - elems;
    if (index > sz) {
      if constexpr (std::forward_iterator<InputIt>) {
        std::size_t count =
            static_cast<std::size_t>(std::distance(first, last));
        if (index >= cap)
          reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + count));
      } else {
        if (index >= cap)
          reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + 1));
      }
      for (std::size_t i = sz; i < index; ++i) {
        std::allocator_traits<Allocator>::construct(alloc, elems + i);
      }
      sz = index;
    }
    if constexpr (std::forward_iterator<InputIt>) {
      std::size_t count = static_cast<std::size_t>(std::distance(first, last));
      if (count == 0)
        return const_cast<iterator>(pos);
      if (sz + count > cap)
        reserve(std::max(sz ? sz * _VECTOR_GROW : 1, sz + count));
      if constexpr (std::is_trivially_copyable_v<T>) {
        std::memmove(elems + index + count, elems + index,
                     (sz - index) * sizeof(T));
        if constexpr (std::contiguous_iterator<InputIt>)
          std::memmove(elems + index, &*first, count * sizeof(T));
        else
          std::copy(first, last, elems + index);
      } else {
        for (std::size_t i = sz + count - 1; i >= index + count; --i) {
          if (i < sz)
            elems[i] = std::move(elems[i - count]);
          else
            std::allocator_traits<Allocator>::construct(
                alloc, elems + i, std::move(elems[i - count]));
        }
```

```
      std::size_t i = index;
      for (; i < index + count && first != last; ++i, ++first) {
        if (i < sz)
          elems[i] = *first;
        else
          std::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                      *first);
      }
    }
    sz += count;
    return elems + index;
  } else {
    vector<T, Allocator> tmp(first, last, alloc);
    return insert(pos, tmp.begin(), tmp.end());
  }
}

constexpr iterator insert(const_iterator pos,
                          std::initializer_list<T> ilist) {
  return insert(pos, ilist.begin(), ilist.end());
}

template <class Arg>
constexpr iterator emplace(const_iterator pos, Arg &&arg) {
  std::size_t index = pos - elems;
  if (index > sz) {
    if (index >= cap)
      reserve(std::max(sz ? sz * _VECTOR_GROW : 1, index + 1));
    for (std::size_t i = sz; i < index; ++i) {
      std::allocator_traits<Allocator>::construct(alloc, elems + i);
    }
    sz = index;
  } else if (sz >= cap)
    reserve(sz ? sz * _VECTOR_GROW : 1);
  if (index == sz) {
    if constexpr (std::is_trivially_move_constructible_v<T>)
      elems[sz] = T(std::forward<Arg>(arg));
    else
      std::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::forward<Arg>(arg));
  } else {
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::memmove(elems + index + 1, elems + index,
                   (sz - index) * sizeof(T));
      elems[index] = T(std::forward<Arg>(arg));
    } else {
      std::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::move(elems[sz - 1]));
      for (std::size_t i = sz - 1; i > index; --i)
        elems[i] = std::move(elems[i - 1]);
      elems[index] = T(std::forward<Arg>(arg));
    }
  }
  ++sz;
  return elems + index;
}

constexpr iterator erase(const_iterator pos) {
  std::size_t index = pos - elems;
  if (index >= sz)
    return elems + sz;
  if constexpr (std::is_trivially_copyable_v<T>) {
```

```
      std::memmove(elems + index, elems + index + 1,
                   (sz - index - 1) * sizeof(T));
    } else {
      for (std::size_t i = index; i < sz - 1; ++i)
        elems[i] = std::move(elems[i + 1]);
      std::allocator_traits<Allocator>::destroy(alloc, elems + sz - 1);
    }
    --sz;
    return elems + index;
  }

  constexpr iterator erase(const_iterator first, const_iterator last) {
    if (first == last)
      return const_cast<iterator>(first);
    std::size_t index = first - elems;
    std::size_t end_index = last - elems;
    if (index >= sz)
      return elems + sz;
    if (end_index > sz)
      end_index = sz;
    std::size_t count = end_index - index;
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::memmove(elems + index, elems + index + count,
                   (sz - index - count) * sizeof(T));
    } else {
      for (std::size_t i = index; i + count < sz; ++i)
        elems[i] = std::move(elems[i + count]);
      for (std::size_t i = sz - count; i < sz; ++i)
        std::allocator_traits<Allocator>::destroy(alloc, elems + i);
    }
    sz -= count;
    return elems + index;
  }

  constexpr void push_back(const T &value) {
    if (sz >= cap)
      reserve(sz ? sz * _VECTOR_GROW : 1);
    if constexpr (std::is_trivially_copyable_v<T>)
      elems[sz++] = value;
    else {
      std::allocator_traits<Allocator>::construct(alloc, elems + sz, value);
      ++sz;
    }
  }

  constexpr void push_back(T &&value) {
    if (sz >= cap)
      reserve(sz ? sz * _VECTOR_GROW : 1);
    if constexpr (std::is_trivially_move_constructible_v<T>)
      elems[sz++] = std::move(value);
    else {
      std::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::move(value));
      ++sz;
    }
  }

  template <class Arg> constexpr T &emplace_back(Arg &&arg) {
    if (sz >= cap)
      reserve(sz ? sz * _VECTOR_GROW : 1);
    if constexpr (std::is_trivially_move_constructible_v<T>)
      elems[sz] = T(std::forward<Arg>(arg));
```

```
    else
      std::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::forward<Arg>(arg));
    return elems[sz++];
  }

  constexpr void pop_back() {
    --sz;
    if constexpr (!std::is_trivially_destructible_v<T>)
      std::allocator_traits<Allocator>::destroy(alloc, elems + sz);
  }

  constexpr void resize(std::size_t new_size) {
    if (new_size < sz) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = new_size; i < sz; ++i)
          std::allocator_traits<Allocator>::destroy(alloc, elems + i);
      sz = new_size;
      return;
    }
    if (new_size > cap)
      reserve(new_size);
    if (new_size > sz) {
      if constexpr (std::is_trivially_default_constructible_v<T>) {
        std::uninitialized_default_construct(elems + sz, elems + new_size);
      } else if constexpr (std::is_default_constructible_v<T>) {
        for (std::size_t i = sz; i < new_size; ++i)
          std::allocator_traits<Allocator>::construct(alloc, elems + i);
      } else {
        throw std::logic_error("vector");
      }
      sz = new_size;
    }
  }

  constexpr void resize(std::size_t new_size, const T &value) {
    if (new_size < sz) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = new_size; i < sz; ++i)
          std::allocator_traits<Allocator>::destroy(alloc, elems + i);
      sz = new_size;
      return;
    }
    if (new_size > cap)
      reserve(new_size);
    if (new_size > sz) {
      if constexpr (std::is_trivially_default_constructible_v<T>) {
        std::uninitialized_fill(elems + sz, elems + new_size, value);
      } else {
        for (std::size_t i = sz; i < new_size; ++i)
          std::allocator_traits<Allocator>::construct(alloc, elems + i, value);
      }
      sz = new_size;
    }
  }

  constexpr void swap(vector &other) noexcept(
      std::allocator_traits<Allocator>::propagate_on_container_swap::value ||
      std::allocator_traits<Allocator>::is_always_equal::value) {
    if constexpr (std::allocator_traits<
                      Allocator>::propagate_on_container_swap::value)
      std::swap(alloc, other.alloc);
```

```
    std::swap(elems, other.elems);
    std::swap(sz, other.sz);
    std::swap(cap, other.cap);
  }
};

template <class InputIt,
          class Allocator =
              std::allocator<typename std::iterator_traits<InputIt>::value_type>>
vector(InputIt, InputIt, Allocator = Allocator())
    -> vector<typename std::iterator_traits<InputIt>::value_type, Allocator>;

template <class T, class Allocator>
constexpr void
swap(std::vector<T, Allocator> &lhs,
     std::vector<T, Allocator> &rhs) noexcept(noexcept(lhs.swap(rhs))) {
  lhs.swap(rhs);
}

template <class T, class Allocator, class U>
constexpr typename std::vector<T, Allocator>::size_type
erase(std::vector<T, Allocator> &c, const U &value) {
  auto it = std::remove(c.begin(), c.end(), value);
  auto r = c.end() - it;
  c.erase(it, c.end());
  return r;
}

template <class T, class Allocator, class Pred>
constexpr typename std::vector<T, Allocator>::size_type
erase_if(std::vector<T, Allocator> &c, Pred pred) {
  auto it = std::remove_if(c.begin(), c.end(), pred);
  auto r = c.end() - it;
  c.erase(it, c.end());
  return r;
}

template <class T, class Allocator>
constexpr bool operator==(const std::vector<T, Allocator> &lhs,
                          const std::vector<T, Allocator> &rhs) {
  if (lhs.size() != rhs.size())
    return false;
  return std::equal(lhs.begin(), lhs.end(), rhs.begin());
}

template <class T, class Allocator>
constexpr std::strong_ordering
operator<=>(const std::vector<T, Allocator> &lhs,
            const std::vector<T, Allocator> &rhs) {
  return std::lexicographical_compare_three_way(
      lhs.begin(), lhs.end(), rhs.begin(), rhs.end(), std::compare_three_way());
}
```

VII Singly linked list

A singly linked list is an unordered list where each node stores only the next node. It has O(1) peeking, pushing, and popping front and insertion after node in O(1) time, but random access, removal of node, and insertion before node in O(n) time. If the tail is

stored, it has peeking and pushing back in O(1) time. It is a stack with head as top and tail as bottom. If the tail is stored, it is a queue with the tail as front and head as back.

An implementation in Python:

```
class _SNode:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next


class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.head is None

    def __getitem__(self, num):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError("list index out of range")
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        return cur.data

    def __setitem__(self, num, x):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError("list index out of range")
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        cur.data = x

    def push_front(self, x):  # O(1)
        self.head = _SNode(x, self.head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _SNode(x)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError("remove from empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
```

```
            self.tail = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.tail.data

    def insert_after_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.tail:
            return self.push_back(x)
        node = _SNode(x, pos.next)
        pos.next = node
        self.size += 1

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next
```

VIII Doubly linked list

A doubly linked list is an unordered list where each node stores the previous node and the next node. It has peeking, pushing, and popping front and back, insertion after and before node, and removal of node in O(1) time, but random access in O(n) time. It is a deque.

An implementation in Python:

```
class _DNode:
    def __init__(self, data, prev=None, next=None):
        self.data = data
        self.prev = prev
        self.next = next


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

```

```
    def is_empty(self):  # O(1)
        return self.size == 0

    def __getitem__(self, num):  # O(min(n, size - n)), n = num if num > 0 else num + size
        if not -self.size <= num < self.size:
            raise IndexError("list index out of range")
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        return cur.data

    def __setitem__(self, num, x):  # O(min(n, size - n)), n = num if num > 0 else num + size
        if not -self.size <= num < self.size:
            raise IndexError("list index out of range")
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        cur.data = x

    def push_front(self, x):  # O(1)
        node = _DNode(x, None, self.head)
        if self.head is None:
            self.head = self.tail = node
        else:
            self.head.prev = node
            self.head = node
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _DNode(x, self.tail, None)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError("pop from empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
```

```
            self.tail = None
        else:
            self.head.prev = None
        return x

    def pop_back(self):  # O(1)
        if self.tail is None:
            raise IndexError("pop from empty list")
        x = self.tail.data
        self.tail = self.tail.prev
        self.size -= 1
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek at empty list")
        return self.tail.data

    def insert_after_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.tail:
            return self.push_back(x)
        node = _DNode(x, pos, pos.next)
        pos.next.prev = node
        pos.next = node
        self.size += 1

    def insert_before_node(self, pos, x):  # O(1)
        if self.is_empty():
            raise IndexError("insert at empty list")
        if pos is self.head:
            return self.push_front(x)
        node = _DNode(x, pos.prev, pos)
        pos.prev.next = node
        pos.prev = node
        self.size += 1

    def remove_node(self, node):  # O(1)
        if self.head is None:
            raise IndexError("remove from empty list")
        if self.head is node:
            return self.pop_front()
        elif self.tail is node:
            return self.pop_back()
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1
        return node.data

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
```

```
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next

    def __reversed__(self):
        cur = self.tail
        while cur is not None:
            yield cur.data
            cur = cur.prev
```

IX Deque

A doubly-ended queue (deque) is a list that is simultaneously a stack and a queue, that is, has peeking, pushing, and popping from both ends in O(1) time.

An implementation of deque is a dynamic array of fixed-size blocks, which has random access and popping front and back in O(1) time, pushing front and back in amortized O(1) time, but removal and insertion in O(n) time. A such implementation in Python:

```
class BlockListDeque:  # Both ends IO amortized O(1)
    def __init__(self, block_size=64, front_step=1, back_step=1):
        if block_size <= 0:
            raise ValueError("deque block_size must >0")
        if front_step <= 0:
            raise ValueError("deque front_step must >0")
        if back_step <= 0:
            raise ValueError("deque back_step must >0")
        self.block_size = block_size
        self.front_step = front_step
        self.back_step = back_step
        self.data = [[None] * self.block_size for _ in range(self.front_step + self.back_step - 1)]
        self.block_begin = self.front_step - 1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1) // 2
        self.in_end = self.in_begin
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.size == 0

    def capacity(self):  # O(1)
        return self.block_size * len(self.data)

    def is_valid(self):  # O(1)
        return self.size == self.block_size * (self.block_end - self.block_begin - 1) + self.in_end
            - self.in_begin

    def __getitem__(self, index):  # O(1)
        if not -self.size <= index < self.size:
            raise IndexError("deque index out of range")
        if index < 0:
```

```
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        return self.data[self.block_begin + block_offset][in_block]

    def __setitem__(self, index, x):  # O(1)
        if not -self.size <= index < self.size:
            raise IndexError("deque index out of range")
        if index < 0:
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        self.data[self.block_begin + block_offset][in_block] = x

    def reserve_front(self, blocks=None):  # O(1)
        if blocks is None:
            blocks = self.front_step
        if blocks <= self.block_begin:
            return
        new = [[None] * self.block_size for _ in range(blocks)]
        new.extend(self.data)
        self.data = new
        self.block_begin += blocks
        self.block_end += blocks

    def reserve_back(self, blocks=None):  # O(1)
        if blocks is None:
            blocks = self.back_step
        if self.block_end + blocks <= len(self.data):
            return
        self.data.extend([[None] * self.block_size for _ in range(blocks)])

    def shrink_front_to_fit(self):
        new = self.data[self.block_begin :]
        self.data = new
        self.block_end -= self.block_begin
        self.block_begin = 0

    def shrink_back_to_fit(self):
        for _ in range(self.block_end, len(self.data)):
            self.data.pop()

    def push_front(self, x):  # amortized O(1)
        if self.in_begin != 0:
            self.in_begin -= 1
        else:
            if self.block_begin == 0:
                self.reserve_front()
            self.block_begin -= 1
            self.in_begin = self.block_size - 1
        self.data[self.block_begin][self.in_begin] = x
        self.size += 1

    def push_back(self, x):  # amortized O(1)
        if self.in_end != self.block_size:
            self.in_end += 1
        else:
            if self.block_end == len(self.data):
```

```
                self.reserve_back()
            self.block_end += 1
            self.in_end = 1
        self.data[self.block_end - 1][self.in_end - 1] = x
        self.size += 1

    def pop_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("pop from empty deque")
        x = self.data[self.block_begin][self.in_begin]
        self.data[self.block_begin][self.in_begin] = None
        self.size -= 1
        if self.in_begin != self.block_size - 1:
            self.in_begin += 1
        else:
            self.block_begin += 1
            self.in_begin = 0
        return x

    def pop_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("pop from empty deque")
        x = self.data[self.block_end - 1][self.in_end - 1]
        self.data[self.block_end - 1][self.in_end - 1] = None
        self.size -= 1
        if self.in_end != 1:
            self.in_end -= 1
        else:
            self.block_end -= 1
            self.in_end = self.block_size
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek from empty deque")
        return self.data[self.block_begin][self.in_begin]

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError("peek from empty deque")
        return self.data[self.block_end - 1][self.in_end - 1]

    def clear(self):  # O(len(self.data))
        num_blocks = len(self.data)
        self.data = [[None] * self.block_size for _ in range(num_blocks)]
        self.block_begin = self.front_step - 1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1) // 2
        self.in_end = self.in_begin
        self.size = 0

    def __iter__(self):
        block = self.block_begin
        index = self.in_begin
        for _ in range(self.size):
            yield self.data[block][index]
            index += 1
            if index == self.block_size:
                index = 0
                block += 1

    def __reversed__(self):
```

```
        block = self.block_end - 1
        index = self.in_end - 1
        for _ in range(self.size):
            yield self.data[block][index]
            index -= 1
            if index < 0:
                index = self.block_size - 1
                block -= 1
```